

SignalScope Integration Report

Date: 2026-09-11

Scope

This report summarizes the current end-to-end integration between the Django backend, the PyTorch/ResNet18 inference path, and the Next.js frontend.

Connected Paths

- Frontend calls GET /api/ping/ to verify backend availability on load.
- Frontend submits multipart image uploads to POST /api/predict/ and renders the returned verdict and probabilities.
- Django view code forwards image bytes into detector.services.classify_image, which loads a checkpointed PyTorch ResNet18 model when available.
- If the checkpoint is missing, the backend now serves a development baseline and marks the response as untrained instead of returning 503.
- The explain endpoint reuses the same inference path and appends safe limitation text for development use.

Validation Results

Check	Result
pytest backend/detector/tests/test_api.py tests/test_ml_pipeline.py -q	passed
cd frontend && npm run build	Succeeded

Current Limitation

A trained checkpoint still remains the preferred deployment path for meaningful detection quality. The fallback keeps the application online for development and integration testing.

Primary Files

- backend/detector/services.py
- backend/detector/views.py
- frontend/src/app/page.tsx
- tests/test_ml_pipeline.py